

1 · 순차 데이터 & RNN 핵심 아이디어

순차 데이터(sequential data): 입력이 시퀀스 $\{x_1, x_2, \dots, x_n\}$. 예) 언어 · 비디오 · 시계열 · 오디오.
RNN = 시퀀스를 처리하며 갱신되는 **내부 상태 h_t (hidden state)**를 가진 구조.
 핵심: **모든 시점에서 같은 함수 f 와 같은 파라미터 W 를 공유** (unroll 해도 W 하나).

$$h_t = f_W(h_{t-1}, x_t)$$

새 상태 = 함수(이전 상태, 현재 입력)

$$y_t = f_{W_y}(h_t)$$

출력 = 또 다른 함수(새 상태)

2 · Vanilla RNN 수식 ★암기

$$h_t = \tanh(W_{hh} h_{t-1} + W_{xh} x_t + b_h)$$

$$y_t = W_{hy} h_t + b_y$$

가중치 3종 — W_{xh} : 입력→은닉, W_{hh} : 은닉→은닉(recurrent), W_{hy} : 은닉→출력. 활성화함수는 $\tanh$.

3 · 입·출력 구조 (I/O configurations)

one→one 일반 분류 (시퀀스 X)
one→many 이미지 캡셔닝: 이미지→단어 시퀀스
many→one 감성분석: 단어 시퀀스→감성 클래스
many→many 기계 번역: 언어 A 문장→언어 B 문장

Multilayer RNN: 시간 방향(time-wise)뿐 아니라 **깊이 방향(depth-wise)**으로도 쌓을 수 있음.

4 · One→Many 계산그래프 (입력이 1개일 때)

첫 시점에만 입력 x_1 이후 시점 입력 처리 방식 2가지:

- **Case 1:** 이후 시점 입력으로 **0**(영벡터)을 넣음.
- **Case 2:** 이전 시점 출력 y_{t-1} 을 다음 입력으로 넣음 (자기회귀).

5 · 문자단위 언어모델 (char-level LM) 예제

Vocab [h, e, l, o], 학습열 "hello". 각 문자는 one-hot: $h=[1,0,0,0]$, $e=[0,1,0,0]$, $l=[0,0,1,0]$, $o=[0,0,0,1]$.

출력층 점수에 **softmax** → 다음 문자 확률. **생성(test) 시:** 자신의 **이전 예측**을 다음 입력으로 넣어 한 글자씩 만들어 냄.

6 · RNN Gradient Flow (BPTT) ★핵심

스택 표현: $h_t = \tanh(W[h_{t-1}; x_t])$, 여기서 $W=[W_{hh} \ W_{xh}]$.

$h_t \rightarrow h_{t-1}$ 역전파는 매번 **$\tanh'$ 와 W_{hh} 를 곱함**:

$$\partial h_t / \partial h_{t-1} = \tanh'(W_{hh} h_{t-1} + W_{xh} x_t) \cdot W_{hh}$$

전체 손실: $\partial L / \partial W = \sum_{t=1}^T \partial L_t / \partial W$. 한 항을 펼치면 곱이 길게 누적:

$$\partial L_T / \partial W = (\partial L_T / \partial h_T) \cdot (\prod_{t=2}^T \partial h_t / \partial h_{t-1}) \cdot (\partial h_1 / \partial W)$$

7 · Vanishing / Exploding Gradient ★시험

$\tanh'$ 값은 거의 항상 $< 1 \rightarrow$ 곱이 누적되며 **gradient 소실(vanishing)**.
 비선형 제거 시 W_{hh} 만 반복 곱:

$$\partial L_T / \partial W = (\partial L_T / \partial h_T) \cdot W_{hh}^{T-1} \cdot (\partial h_1 / \partial W)$$

W_{hh} 의 **최대 특이값(largest singular value)** 기준:

> 1 Exploding → 해결: **Gradient clipping** (norm 크면 스케일 ↓)

< 1 Vanishing → 해결: **RNN 구조 변경 (LSTM)**

특이값 = 행렬이 주축 방향으로 벡터를 늘리/줄이는 정도

8 · LSTM (Long Short-Term Memory) ★암기

게이트로 여러 시점에 걸쳐 정보 추적, gate 메커니즘으로 vanishing 완화.
 (σ =sigmoid 0~1, $\odot$ =원소별 곱)

i_t Input: 새 입력을 cell에 얼마나 쓸지

f_t Forget: 이전 cell을 얼마나 지울지

o_t Output: 새 cell을 얼마나 드러낼지

$$i_t = \sigma(W_{xi} x_t + W_{hi} h_{t-1} + b_i)$$

$$f_t = \sigma(W_{xf} x_t + W_{hf} h_{t-1} + b_f)$$

$$o_t = \sigma(W_{xo} x_t + W_{ho} h_{t-1} + b_o)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tanh(W_{xc} x_t + W_{hc} h_{t-1} + b_c)$$

$$h_t = o_t \odot \tanh(c_t)$$

Gradient flow: $c_t \rightarrow c_{t-1}$ 역전파는 **곱셈·덧셈만** 포함 → gradient 흐름 개선 (ResNet과 유사) → vanishing 완화.

9 · Seq2Seq : Encoder-Decoder

Naive many→many 문제: **입력 길이 = 출력 길이(1:1)**를 가정 → 언어마다 문장 길이가 다르고 어순도 다름.

해결: 모델을 **Encoder / Decoder** 둘로 분할.

Encoder 출력층 없는 표준 RNN, 입력을 압축해 마지막 **hidden state**를 Decoder로 전달

Decoder Dense층으로 출력 생성, **<bos>**로 시작 **<eos>**로 종료

학습: teacher forcing(정답 토큰을 디코더 입력) / **추론: 자신의 이전 예측**을 입력으로.

(Sutskever et al., 2014)

★ 시험 직전 한눈 체크 (요약)

점화식 $h_t = f_W(h_{t-1}, x_t)$ — 모든 시점 **W** 공유

4 구조 $1 \rightarrow 1 \cdot 1 \rightarrow N$ (캡셔닝) · $N \rightarrow 1$ (감성) · $N \rightarrow N$ (번역)

Vanishing $\tanh' < 1 \times W_{hh}$ 반복곱 → 소실

Exploding W_{hh} 최대 특이값 > 1

처방 Exploding→**clipping** / Vanishing→**LSTM**

LSTM cell c_t 의 덧셈 경로로 gradient 보존($\approx$ ResNet)

Seq2Seq Encoder(출력층X)→hidden→Decoder, 학습=**teacher forcing**